

Unit 4 System, Application, and Cloud Security

4.1 System Security (Operating System Security Concepts)

Protection is about controlling access inside the system.

It ensures that users, programs, and processes only use resources (files, memory, CPU) in allowed ways.

Security is about defending the system from outside attacks.

It prevents unauthorized people from entering or damaging the system.

Security can be attacked in the following ways:

1. Authorization: pretending to be a valid user
2. Browsing: accessing data without permission
3. Trap doors: hidden entry points in software
4. Invalid Parameters: giving wrong inputs to crash or exploit systems
5. Line Tapping: intercepting communication (like network spying)
6. Electronic Data Capture: stealing data electronically
7. Lost Line: losing secure communication channels
8. Improper Access Controls: weak permission settings
9. Waste Recovery: retrieving deleted data
10. Rogue Software: malicious programs installed secretly

Security ensures:

- Only authorized users can access data
- Data is not modified illegally
- Data is available when needed

The OS helps users share files safely:

- File owner sets permissions (read, write, execute)
- OS enforces these rules strictly

Security Threats:

Trojan Horse: Fake software hiding malicious intent

Virus: Infects files and spreads

Worm: Self-replicates across networks

Buffer Overflow: Exploits memory errors

Unit 4 System, Application, and Cloud Security

Authentication and Authorization Mechanism

- Modern systems need strong access control to protect sensitive data
- They ensure only authorized users can access specific resources
- They prevent unauthorized access and security breaches
- Authentication verifies the identity of a user or system
- Authorization determines what resources an authenticated user can access
- Together, they form a security perimeter for protecting digital systems

Authentication:

Identity Verification: The process of confirming that a user is who they claim to be.

- Authentication is the entry gate to any secure system.
- It verifies the digital identity of a person, device, or application before allowing any interaction.

Authorization:

Access Permission: The process of determining the specific resources or actions a verified user can access.

Once identity is confirmed, authorization defines the boundaries of that user's activity within the system.

- Grants or denies specific permissions.
- Based on organizational roles or rules
- Example: Admin vs Viewer access

Basis	Authentication	Authorization
Definition	Authentication is the process of verifying the identity of a user, system, or device.	Authorization is the process of determining what resources or actions an authenticated user is allowed to access.
Main Question	"Who are you?"	"What are you allowed to do?"
Purpose	Confirms the identity of users.	Grants or restricts permissions and access rights.
Occurs When	Happens first before access is granted.	Happens after successful authentication.
Methods Used	Username/password, PIN, biometrics, OTP, smart cards.	Access Control Lists (ACL), roles, permissions, policies.
Data Required	User credentials.	User roles and privileges.

Unit 4 System, Application, and Cloud Security

Basis	Authentication	Authorization
Result	User identity is verified.	User receives specific access rights.
Example	Logging into a university portal using username and password.	After login, a student can view grades but cannot modify them.

Access Control Models:

Access control is a security mechanism that manages who is allowed to access information, systems, or even physical spaces. Its main purpose is to ensure that only the right people or authorized processes can gain access, helping to reduce security risks and protect valuable resources.

- Makes sure only verified and approved users can access resources
- Uses both authentication and authorization to manage access permissions
- Helps protect digital systems as well as physical environments
- Strengthens security while supporting compliance and accountability

Types of Access Control (DAC)

The different types of Access Control are as follows:

1. Discretionary Access Control (DAC)

Access is controlled by the owner of the resource.

- Owners decide who gets access
- Flexible but less secure
- Common in personal computers and small systems

2. Mandatory Access Control (MAC)

A central authority enforces access based on strict security levels.

- Used in high-security environments
- Users cannot change permissions
- Example: Military or government systems, SELinux

3. Role Based Access Control (RBAC)

Users receive permissions based on their job roles.

- Simplifies large-scale permission management
- Reduces human error and misuse
- Example: HR role cannot create network accounts

4. Attribute Based Access Control (ABAC)

Access decisions are based on multiple attributes (user, device, environment)

- Highly dynamic and flexible
- Evaluates policies using many attributes

Unit 4 System, Application, and Cloud Security

- Useful in modern cloud and zero-trust systems

5. Rule Based Access Control (Ru-BAC)

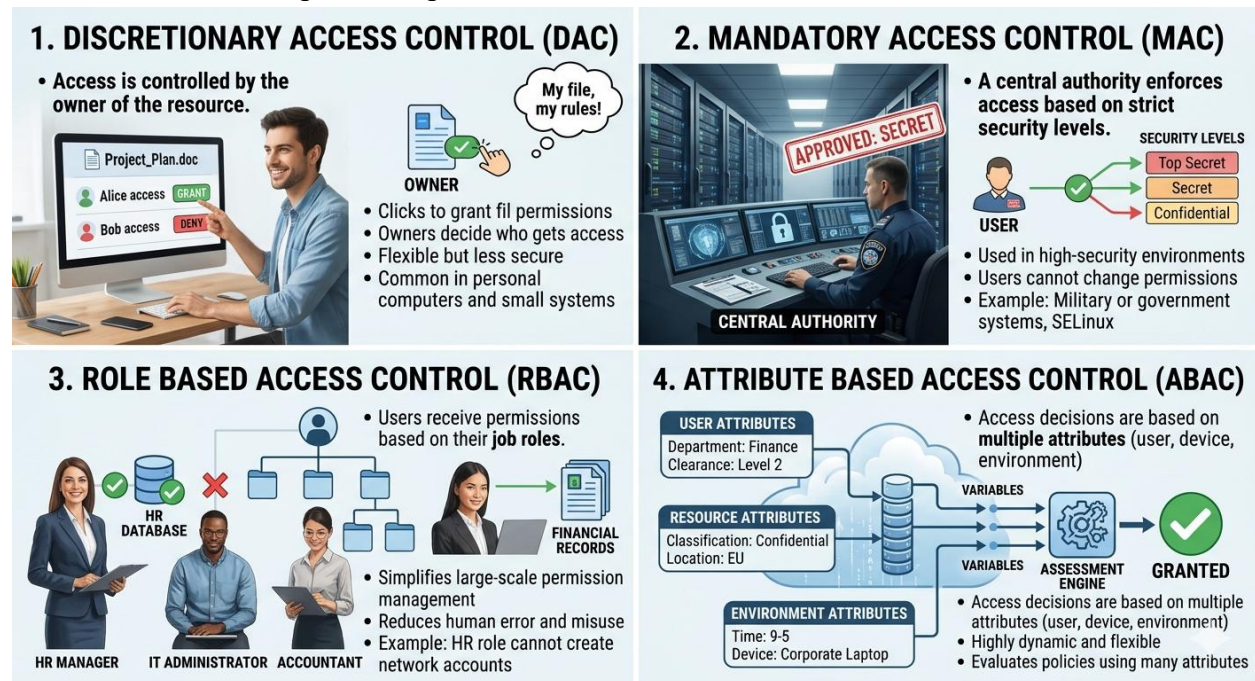
Access is granted based on predefined rules or conditions.

- Often time-based or context-based
- System automatically enforces rules
- Example: Students allowed lab access only during class hours

6. Organization Based Access Control (Or-BAC)

Access policies are defined independently of the technical system.

- Focuses on organizational rules
- Separates policy design from implementation
- Useful for complex enterprise environments



Secure Boot and trusted Computing

In system security, protecting a computer only after it starts running is not enough. Security should begin from the very first moment a device is powered on. Two important concepts that help achieve this are **Secure Boot** and **Trusted Computing**. These technologies work together to make systems safer and more resistant to attacks.

Secure Boot

Secure Boot is a security feature that helps ensure a computer starts only with trusted and approved software. During the startup process, a computer loads several programs before the operating system begins running. If an attacker inserts malicious code into this startup sequence, the entire system can become compromised.

Secure Boot prevents this problem by checking the digital signature of boot components before they are executed. The system verifies whether the software comes from a trusted source. If the

Unit 4 System, Application, and Cloud Security

software is authentic and approved, the boot process continues. If unknown or modified code is detected, the system blocks it from loading.

In simple terms, Secure Boot acts like a security guard at the entrance of a building. It checks the identity of everyone entering and allows only trusted individuals inside.

Main purposes of Secure Boot:

Main purposes of Secure Boot:

- Prevents malware from loading during startup
- Protects against bootkits and rootkits
- Ensures only trusted software runs during system boot
- Maintains the integrity of the startup process

Example:

When you turn on a laptop, Secure Boot checks whether the operating system and startup files are genuine. If malware has secretly modified the boot files, Secure Boot can stop the system from loading them.

Trusted Computing:

Trusted Computing is a broader security approach designed to create a reliable and secure computing environment. It uses both hardware and software technologies to ensure that a system behaves as expected and has not been altered by unauthorized users or malicious programs.

A key component of Trusted Computing is the Trusted Platform Module (TPM). TPM is a specialized hardware chip that securely stores cryptographic keys, passwords, and security information. It can verify system integrity and support security functions such as encryption and secure authentication.

Trusted Computing helps establish trust in a system by verifying that hardware, software, and system components remain unchanged and secure.

Think of Trusted Computing as building a house with strong locks, security cameras, and identity checks. Instead of protecting only one door, it creates multiple layers of protection throughout the system.

Main goals of Trusted Computing:

- Creates a secure and trustworthy computing environment
- Verifies system integrity and identity
- Uses hardware-based security features
- Protects sensitive information and cryptographic keys
- Reduces the risk of system tampering and attacks

Unit 4 System, Application, and Cloud Security

Example:

A company laptop may use TPM and Trusted Computing technologies to verify that the operating system has not been modified. If unauthorized changes are detected, access to sensitive data can be restricted.

Relationship Between Secure Boot and Trusted Computing

Secure Boot and Trusted Computing complement each other. Secure Boot focuses on protecting the startup process, while Trusted Computing provides a broader framework for maintaining trust and security throughout system operation.

Secure Boot helps ensure that the system starts safely, while Trusted Computing helps ensure it remains trustworthy afterward.

Together, they provide stronger system protection and improve overall security.

4.2 Application Security:

Secure Software Development Lifecycle (SSDLC)

In software security, building a secure application should not be treated as an activity done only at the end of development. Security should be considered from the beginning and followed throughout the entire development process. This idea is known as the **Secure Software Development Lifecycle (SSDLC)**.

SSDLC is a process that integrates security practices into every phase of software development. Instead of finding problems after the software is completed, developers identify and reduce security risks during planning, design, coding, testing, and maintenance stages.

In simple terms, SSDLC is like building a house. If safety features such as strong doors, locks, and security cameras are planned from the beginning, the house becomes safer. Adding them only after construction is completed is usually more difficult and expensive.

OWASP Top 10 Vulnerabilities:

- OWASP Top 10 is a list of the most critical web application security risks
- Created by the Open Web Application Security Project (OWASP)
- Based on real-world security data and industry feedback
- Used as a global standard for application security awareness
- Helps developers identify and prevent common vulnerabilities
- Improves secure coding and application safety

The top 10 OWASP are as follows:

1. Broken Access Control

- Users can access data or features they should not have permission for
- Happens when authorization rules are not properly enforced
- Can lead to unauthorized data access or actions
- Example: normal user accessing admin pages or data

Unit 4 System, Application, and Cloud Security

2. Injection (SQL, Command Injection)

- Occurs when untrusted input is sent to a system or database
- Attackers can insert malicious commands through input fields
- Can lead to data theft, modification, or system control
- Example: SQL injection to bypass login or extract data

3. Authentication Failures

- Weak or broken login and identity verification systems
- Users can be easily impersonated or accounts taken over
- Caused by weak passwords, no MFA, or poor session handling
- Example: brute-force login due to no account lockout or MFA

4. Security Misconfiguration

- Incorrect or insecure system, server, or application settings
- Often caused by default configurations or unused features left enabled
- Can expose sensitive data or admin access
- Example: default passwords, open admin panels, or public cloud storage

5. Sensitive Data Exposure (Cryptographic Failures)

- Sensitive data is not properly protected or encrypted
- Data can be read if intercepted or accessed illegally
- Caused by weak or missing encryption methods
- Example: storing passwords in plain text or using weak encryption

6. Vulnerable and Outdated Components

- Happens when an application uses old or unpatched software, libraries, or frameworks
- Occurs when developers don't update dependencies or ignore security patches
- Can allow attackers to exploit known security flaws in those components
- Example: A website uses an old version of a framework that has a known security bug, and attackers exploit it to break into the system

7. Insecure Design

- Happens when the system is built without proper security thinking from the start
- Occurs when security is not included in the design phase, only added later
- Can allow attackers to exploit flaws in the system's logic or architecture, not just bugs
- Example: A banking app allows unlimited password reset attempts without rate limiting, making brute-force attacks easy

Unit 4 System, Application, and Cloud Security

8. Software and Data Integrity Failures

- Happens when software or data is not properly checked to make sure it is safe and unchanged
- Occurs when applications trust updates, files, or external code without verifying their authenticity
- Can lead to malware installation, system compromise, or corrupted data
- Example: A website automatically installs a software update, but the update is modified by an attacker and contains malicious code

9. Security Logging and Monitoring Failures

- Happens when a system does not properly record or track important security events
- Occurs when logs are missing, incomplete, or not monitored regularly
- Attackers can stay inside the system without being detected
- Example: A hacker tries multiple password attempts, but the system does not log or alert it

10. Server-side Request Forgery

- Happens when an attacker tricks a server into sending requests to unintended systems (places it should not visit)
- Can allow access to internal services that are not publicly exposed
- Can lead to data exposure, internal system access, or service abuse
- Example: A user enters a URL, and the server is forced to open localhost or internal admin endpoints instead of a normal website.

OWASP Top 10 helps developers understand the most common ways web applications get hacked. It acts as a guideline to build safer and more secure applications. It is used worldwide by developers, companies, and security teams as a standard reference for application security.

Code Injection and Buffer Overflow

Imagine you have built a software application that accepts input from users. Users enter their names, passwords, comments, or upload files. Normally, the software processes this information as data.

But what happens if an attacker tricks the software into treating their input as executable code?

This is where **Code Injection** and **Buffer Overflow Attacks** come into play.

These attacks exploit weaknesses in software design and programming practices, allowing attackers to:

- Execute malicious commands
- Gain unauthorized access
- Steal sensitive information
- Crash systems

Unit 4 System, Application, and Cloud Security

- Take complete control of a computer or server
Many major cyber incidents have occurred because of these vulnerabilities.

Code Injection:

Code Injection is a cyber-attack in which an attacker inserts malicious code into an application so that the application executes it as legitimate code. In simple word, the attacker tricks the program into running commands that were never intended by the developer.

Example: Suppose a website asks for your username:

Enter Username:

A normal user enters: Sunil

An attacker enters: admin'; DROP TABLE Users; --

If the application directly inserts this input into a database query, the database may execute:

***SELECT * FROM Users WHERE Username='admin';
DROP TABLE Users;***

From this above query, the user database gets deleted and system becomes unavailable.

Types of Code Injection Attacks

1. SQL Injection: It targets databases. Example: ***SELECT * FROM users
WHERE username=""
OR '1'='1';***

is always true, authentication may be bypassed.

Effects of SQL Injection:

- Database theft
- Unauthorized access
- Data modification
- Data deletion

2. Command Injection: Targets operating system commands.

Example of vulnerable code: ***system("ping " . \$_GET['host']);***

Attacker input: ***8.8.8.8 && rm -rf ****

Program executes: ping 8.8.8.8

rm -rf *

From this above command, files may be deleted and server may be compromised.

Unit 4 System, Application, and Cloud Security

3. Cross-Site Scripting (XSS): Attacker injects javaScript into website.

Example: `<script>alert("Hacked")</script>`

When other users open the page:

- Script executes in their browser.
- Cookies may be stolen.
- Sessions may be hijacked.

4. **LDAP Injection:** Targets directory services. Example: `*)(uid=*)`

May bypass authentication or expose user records.

Why does code Injection Occur?

1. Improper Input Validation
2. Dynamic Query Construction
3. Lack of Input Sanitization
4. Preventing Code Injection

Buffer: A buffer is a temporary memory area used to store data.

Example: Suppose a program reserves memory for 10 characters.

Buffer size: 10bytes

Buffer overflow: A buffer overflow occurs when a program writes more data into a buffer than it can hold. The extra data spills into adjacent memory locations.

Example: **Morris Worm (1988)**

One of the earliest internet worms exploited a buffer overflow vulnerability. Thousands of systems infected and significant disruption of the early internet.

Unsafe functions in C/C++:

gets()
strcpy()
sprintf()
scanf()

These functions may not check input size.

Types of Buffer Overflow:

1. **Stack-Based Buffer Overflow:** It occurs in stack memory. It returns address and function pointers
2. **Heap-based Buffer overflow:** It occurs in heap memory. It can corrupt dynamic memory structures and object pointers.

Unit 4 System, Application, and Cloud Security

Secure Coding Practices:

Secure coding refers to the practice of writing software in a way that minimizes vulnerabilities and protects applications from malicious attacks.

Secure coding is the process of developing software that is resistant to security threats by following security principles, best practices, and coding standards throughout the software development lifecycle.

Importance of Secure Coding Practices:

Modern applications handle sensitive information such as:

- Usernames and passwords
- Financial transactions
- Personal information
- Medical records
- Business data

Principles of Secure Coding

1. **Principle of Least Privilege:** Users and applications should have only the minimum permissions required. Example: A student account should not have administrator privileges.

2. **Defense in Depth:** Use multiple security layers.

Example:

Firewall
+
Authentication
+
Encryption
+
Input Validation

3. **Keep Security Simple:** Complex security mechanisms often introduce mistakes. Simple solutions are easier to manage and audit.

4.3 Cloud & IoT Security

Cloud Security Challenges and Threats:

Cloud computing has transformed the way organizations store data, run applications, and deliver services. Instead of maintaining physical servers and infrastructure, businesses can use cloud platforms to access computing resources over the Internet. Cloud computing offers numerous benefits such as scalability, flexibility, cost savings, and easy accessibility.

Unit 4 System, Application, and Cloud Security

However, as organizations increasingly move their data and applications to the cloud, security becomes a major concern. Unlike traditional systems where organizations have complete control over their infrastructure, cloud environments introduce new security complexities. Protecting sensitive information, ensuring privacy, and maintaining system availability are some of the major challenges faced by cloud users and providers.

Cloud security challenges refer to the difficulties organizations encounter while protecting cloud-based resources, data, and services from unauthorized access, misuse, and cyberattacks.

Traditional security models were designed for systems located within an organization's physical premises. In cloud environments, data, applications, and services are distributed across multiple locations and managed by third-party providers. This creates additional risks and makes security management more complex.

Some reasons why cloud security is challenging include:

- Shared infrastructure among multiple users.
- Remote access from anywhere in the world.
- Large amounts of sensitive data stored online.
- Complex cloud architectures.
- Rapid deployment of resources.
- Dependence on cloud service providers.

Major Cloud Security Challenges

1. Data Privacy and Confidentiality

Cloud providers store data on remote servers that may be located in different geographical regions. Organizations often have limited visibility into where their data is stored and who can access it.

Example: A hospital stores patient records in the cloud. If unauthorized individuals gain access to those records, confidential medical information may be exposed.

2. Data Loss and Data Leakage

Data loss occurs when information becomes permanently unavailable, while data leakage refers to unauthorized exposure of sensitive information.

Example: A company accidentally deletes critical customer databases without proper backups, resulting in permanent loss of information.

3. Identity and Access Management

Cloud environments may have thousands of users, applications, and devices accessing resources simultaneously. Poor access management can lead to weak passwords, excessive user privileges, and unauthorized access.

Unit 4 System, Application, and Cloud Security

Example: An employee who only needs to view reports is mistakenly given administrator privileges and accidentally deletes important resources.

4. Multi-Tenancy Risks

Multi-tenancy is a cloud architecture where multiple customers share the same physical infrastructure.

Although users are logically separated, they still share underlying hardware resources.

Example: Several companies use the same cloud server. A security flaw allows one company's application to access another company's data.

5. Compliance and Regulatory Requirements

Cloud data may be stored across multiple countries, each with different legal requirements.

Example: A financial institution stores customer information in regions that do not comply with local data protection laws.

6. Lack of Visibility and Control

Organizations often have limited control over cloud infrastructure because it is managed by cloud providers.

Example: An organization suspects suspicious activity but cannot directly inspect the cloud provider's infrastructure.

7. Cloud Misconfiguration

Misconfiguration occurs when cloud resources are improperly configured, creating security vulnerabilities.

Cloud platforms offer numerous configuration options, and mistakes can easily occur.

Example: A company accidentally makes its cloud storage publicly accessible, exposing confidential customer data.

8. Secure Data Storage and Backup Management

Organizations must ensure that cloud data is stored securely and can be recovered if needed.

Example: A ransomware attack encrypts cloud data, and the organization cannot recover it because backups were not properly maintained.

9. Vendor Lock-In

Vendor lock-in occurs when an organization becomes heavily dependent on a particular cloud provider.

Example: An organization builds its entire infrastructure using provider-specific services and later struggles to switch providers.

Unit 4 System, Application, and Cloud Security

Cloud Service models and security (IaaS, PaaS, SaaS)

Imagine that you want to build and run a software application. Traditionally, you would need to purchase servers, install operating systems, configure networks, manage storage, and maintain the entire infrastructure. This process can be expensive, time-consuming, and complex.

Cloud computing solves this problem by providing computing resources over the Internet. Depending on the user's needs, cloud providers offer different levels of services. These service offerings are known as **Cloud Service Models**.

A cloud service model defines **what services are provided by the cloud provider and what responsibilities remain with the customer**.

The three primary cloud service models are:

1. **Infrastructure as a Service (IaaS)**
2. **Platform as a Service (PaaS)**
3. **Software as a Service (SaaS)**

1. Infrastructure as a Service (IaaS)

Infrastructure as a Service (IaaS) is a cloud service model that provides virtualized computing resources over the internet.

In IaaS, the cloud provider offers:

- Virtual machines
- Storage
- Networking
- Servers
- Virtualization technology

The customer manages:

- Operating systems
- Applications
- Data
- Security configurations

Components of IaaS

1. Virtual Machines (VMs)

A virtual machine acts like a physical computer running inside another computer.

Example:

- Linux VM

Unit 4 System, Application, and Cloud Security

- Windows VM

2. Storage

Provides space to store files, databases, and backups.

3. Networking

Provides:

- IP addresses
- Routers
- Firewalls
- Load balancers

4. Virtualization

Allows multiple virtual machines to run on a single physical server.

Advantages of IaaS

- High flexibility
- Full control over resources
- Easy scalability
- Cost-effective compared to buying hardware

Disadvantages

- Requires technical expertise
- Users manage security configurations
- More administrative effort

2. Platform as a Service (PaaS)

Platform as a Service (PaaS) provides a complete platform for developing, testing, and deploying applications.

The cloud provider manages:

- Infrastructure
- Operating system
- Middleware
- Runtime environment

The user focuses only on:

- Application development
- Application data

Unit 4 System, Application, and Cloud Security

Components of PaaS

Development Tools

Provides programming tools and frameworks.

Examples:

- Java SDK
- Python Environment
- Node.js Runtime

Middleware

Software that connects applications with databases and services.

Database Services

Managed databases provided by cloud vendors.

Runtime Environment

Environment where applications execute.

Advantages:

- Faster application development
- Reduced management burden
- Automatic updates
- Easy collaboration among developers

Disadvantages:

- Less control over infrastructure
- Vendor dependency
- Limited customization

3. Software as a Service (SaaS)

Software as a Service (SaaS) provides ready-to-use software applications over the Internet.

Users simply access the software through:

- Web browsers
- Mobile applications

No installation or maintenance is required.

Unit 4 System, Application, and Cloud Security

Advantages:

- Easy to use
- No installation required
- Low initial cost
- Accessible from anywhere

Disadvantages:

- Limited customization
- Internet dependency
- Data privacy concerns

IoT Security Issues and Countermeasures

Imagine waking up in a smart home where your alarm clock wakes you up, your smart thermostat adjusts the room temperature, your smartwatch tracks your health, and your smart refrigerator reminds you to buy groceries. All these devices are part of the Internet of Things (IoT).

The Internet of Things (IoT) refers to a network of physical devices that are connected to the Internet and can collect, exchange, and process data. These devices include sensors, smart appliances, healthcare devices, industrial machines, connected vehicles, and many more.

While IoT provides convenience, automation, and efficiency, it also introduces significant security risks. Many IoT devices have limited processing power and memory, making it difficult to implement strong security mechanisms. As a result, IoT systems have become attractive targets for cybercriminals.

Why is IoT Security Important?

IoT devices often handle sensitive information such as:

- Personal data
- Financial information
- Health records
- Industrial operational data
- Location information

If these devices are compromised, attackers can:

- Steal confidential data
- Spy on users
- Disrupt services
- Cause physical damage
- Launch large-scale cyberattacks

Therefore, ensuring IoT security is essential for maintaining:

Unit 4 System, Application, and Cloud Security

- **Confidentiality** (protecting data from unauthorized access)
- **Integrity** (preventing unauthorized modification)
- **Availability** (ensuring systems remain operational)
- **Privacy** (protecting personal information)

Major IoT Security Issues and Threats

1. **Weak Authentication and Authorization**
2. **Insecure Communication**
3. **Data Privacy Issues**
4. **Malware and Botnet Attacks**
5. **Firmware and Software Vulnerabilities**
6. **Physical Security Threats**
7. **Denial of Service (DoS) Attacks**
8. **Cloud Security Risks**

IoT Security Countermeasures

1. **Strong Authentication**
2. **Secure Communication**
3. **Access Control Mechanisms**
4. **Regular Software and Firmware Updates**
5. **Network Security Measures**
6. **Secure Device Design**
7. **Data Privacy Protection**
8. **Continuous Monitoring and Incident Response**

Unit 5 Cyber Laws, Digital Forensics & Emerging Trends

5.1 Cyber Laws & Ethics

Cyber Ethics and Professional Responsibilities

In today's digital world, engineers, developers, and cybersecurity professionals are not just building software—they are shaping how people live, communicate, work, and store information.

Because of this, technical skills alone are not enough. Professionals must also follow cyber ethics and professional responsibilities to ensure technology is used safely, fairly, and responsibly.

When we use computers, mobile phones, the internet, and social media every day, we are actually operating inside a digital society called cyberspace. Just like physical society has rules and laws, cyberspace also needs rules to control behavior, prevent misuse, and protect users.

This is where two important concepts come in:

- **Cyber Law** → Legal rules that govern online activities
- **Cyber Ethics** → Moral principles that guide responsible behavior in cyberspace

In simple words:

- Cyber law = *What is legally allowed or not allowed*
- Cyber ethics = *What is morally right or wrong online*

Both are essential for engineers, developers, and cybersecurity professionals.

Why Cyber Laws and Ethics Matter

Imagine there are no rules on the internet:

- Anyone can hack any system
- Fake news spreads freely
- Personal data is stolen easily
- Online fraud increases rapidly